

VIRTUAL MUSEUM AR/VR — TECHNICAL REVIEW

AI-Powered Immersive Museum with WebXR AR/VR, RAG Curator, Voice TTS & Cloud Deployment

Date: September 2026 | **Status:** All Phases 0–6 Complete & Tested

Executive Summary: The Virtual Museum AR/VR project is an end-to-end interactive application featuring 3D exhibit galleries, WebXR Augmented and Virtual Reality, a retrieval-augmented generation (RAG) AI curator, speech synthesis (TTS), smooth 60 FPS performance optimization, and a complete multi-cloud deployment pipeline (Vercel + Railway). All core features, interactive controls, audio features, and documentation have been successfully built and verified.

1. Development Roadmap & Implementation Status

Phase	Module / Focus Area	Key Deliverables	Status
Phase 0	Museum Foundation	Next.js 16 + R3F, 3 Galleries, 15 GLB models, WebXR setup	Completed
Phase 1	RAG Vector Core	SentenceTransformers embeddings, NumPy cosine retrieval, cache	Completed
Phase 2	XAI & Anti-Hallucination	0.50 confidence threshold gate, deterministic refusal, evidence UI	Completed
Phase 3	Personalization & Memory	3-turn sliding window memory, tone controls, recommendation engine	Completed
Phase 4	Full-Stack Wiring	FastAPI /ask endpoint, Pydantic contracts, AICuratorPanel integration	Completed
Phase 5	Voice TTS & AR Overlay	Web Speech API synthesis, Listen button, AR DOM Curator overlay	Completed (NEW)
Phase 6	Cloud Deployment Plan	Vercel frontend + Railway FastAPI deployment guide (Deploy.txt)	Completed (NEW)

2. Complete System Features Catalog (23 Features)

Below is the exhaustive catalog of all implemented application features, including single-sentence operational descriptions for each feature:

Feature Name	Category	Type	Operational Description (1-Line Summary)
AI Curator Voice Output (TTS)	Audio & Speech	NEW	Provides browser-native text-to-speech audio narration of curator answers with real-time Play/Stop controls across desktop and AR modes.
In-AR AI Curator Overlay	AR & WebXR	NEW	Integrates the full RAG AI Curator directly inside the mobile WebXR DOM overlay for interactive exhibit Q&A; during AR sessions.
High-FPS Performance Engine	Graphics & Engine	NEW	Maintains a stable 60 FPS through dynamic DPR capping (1.5 max), frustum culling, and distance-based GLB model lazy loading.
360° Mouse Pointer Lock & WASD	Controls & Cam	NEW	Enables FPS-style desktop navigation with WASD movement keys and 360-degree pointer-lock mouse look for smooth exploration.
White Museum Gallery Ceilings	3D Environment	NEW	Refreshes museum interior ceiling geometry with clean white museum-grade materials for accurate light bounce and visual clarity.

Production Deployment Workflow	DevOps & Cloud	NEW	Provides a comprehensive, beginner-friendly deployment pipeline for Vercel (frontend) and Railway (FastAPI backend).
3D Interactive Galleries	3D Environment	Core	Renders three distinct 3D museum halls (Space, History, Natural History) using Three.js and React Three Fiber.
15 Curated GLB Artifacts	3D Artifacts	Core	Displays high-fidelity 3D museum exhibit models loaded dynamically with bounding box optimization and interaction glow.
Artifact Inspection Focus	Interactive UI	Core	Smoothly transitions the camera to focus on selected artifacts while presenting rich exhibit plaques and details.
WebXR Mobile AR Placement	AR & WebXR	Core	Projects 3D exhibit models into physical space via real-world surface plane detection on Android WebXR Chrome.
AR Touch Gesture Controls	AR & WebXR	Core	Supports single-finger object translation and two-finger pinch/twist gestures for real-time scale and rotation in AR.
WebXR VR Headset Mode	VR & WebXR	Core	Provides immersive virtual reality headset support allowing visitors to step into the virtual museum halls directly.
Interactive 2D Minimap	Navigation UI	Core	Renders a real-time overhead floorplan with visitor tracking dots and click-to-teleport navigation across galleries.
SentenceTransformer Vector RAG	AI & NLP	Core	Encodes museum knowledge base into 384-dimensional dense vectors using all-MiniLM-L6-v2 for semantic search.
Fast NumPy Cosine Retrieval	AI & NLP	Core	Executes lightning-fast vector similarity math in NumPy without external database overhead or network latency.
Local Embedding Cache (.npy)	AI & Backend	Core	Accelerates backend startup time by caching serialized vector arrays with auto-invalidation on knowledge base updates.
0.50 Confidence Threshold Gate	XAI & Security	Core	Evaluates cosine similarity against a strict 0.50 threshold to filter out-of-domain queries before calling the LLM.
Grounded Groq LLM Synthesizer	AI & LLM	Core	Synthesizes curator responses using Groq API strictly constrained to retrieved evidence to eliminate AI hallucinations.
Deterministic Query Refusal	XAI & Security	Core	Returns a standardized polite refusal response when queries fall below confidence thresholds without invoking LLM tokens.
XAI Badges & Evidence View	XAI & UI	Core	Exposes numerical similarity scores and verbatim knowledge base excerpts in the UI for transparent answer verification.
Multi-Tone Response Selector	Personalization	Core	Allows visitors to switch curator response styles dynamically between Educational, Concise, and Friendly tones.
3-Turn Contextual Session Memory	Personalization	Core	Maintains a sliding 3-turn conversation memory window to resolve pronouns and follow-up exhibit questions accurately.
Vector Recommendation Engine	Personalization	Core	Suggests relevant exhibit recommendations ('You Might Also Like') based on vector similarity between exhibit records.

3. Technical Architecture & Key Components

3.1 Voice Output Synthesis Pipeline (Text-to-Speech)

The voice output engine ([`src/utls/voice/textToSpeech.js`](file:///c:/Users/hp/CODEBASE/Projects/Museum ARVR/virtual-museum/src/utls/voice/textToSpeech.js)) delivers seamless audio responses without third-party API dependencies or costs:

- **Speech Synthesis Integration:** Utilizes ``window.speechSynthesis`` and ``SpeechSynthesisUtterance`` for low-latency client-side speech generation.
- **Natural Voice Selection:** Automatically scans available device voices to prioritize natural English voices (e.g., Google US English, Natural English, or Microsoft Online voices).
- **Lifecycle & Cleanup Management:** Automatically cancels active speech when the AI Curator panel is closed, when switching artifacts, or when exiting AR mode.
- **UI State Synchronization:** Drives the **■ Listen / ■ Stop** toggle button in ``AICuratorPanel.jsx`` using ``onstart``, ``onend``, and ``onerror`` event listeners.

3.2 WebXR AR Overlay & Exhibit Context Integration

The AR overlay system ([`src/components/ar/AROverlayUI.jsx`](file:///c:/Users/hp/CODEBASE/Projects/Museum ARVR/virtual-museum/src/components/ar/AROverlayUI.jsx)) bridges 3D WebXR rendering with modern 2D DOM interaction:

- **DOM Portal Injection:** Leverages React Portals (``createPortal``) to project HTML UI elements into the WebXR AR overlay container.
- **In-AR AI Curator Panel:** Embeds the full RAG AI Curator inside AR overlay, enabling visitors to ask questions about placed 3D artifacts and listen to voice answers.
- **Gesture-Safe Interaction:** Touch inputs inside the curator panel are isolated from 3D object manipulation to prevent accidental movement during reading or audio playback.

3.3 Performance & FPS Optimization Architecture

To ensure fluid 60 FPS performance on both desktop GPUs and mobile AR hardware, key rendering optimizations were implemented:

- **Dynamic DPR Cap:** Device pixel ratio is hard-capped at 1.5 (``dpr=[1, 1.5]``) in R3F Canvas to prevent rendering overhead on 4K / high-density mobile screens.
- **Camera Frustum Culling:** Mesh geometry outside the active camera field of view is culled automatically prior to rasterization.
- **Distance-Based Lazy Loading:** Heavy GLB artifact geometry loads on demand as the visitor approaches exhibit pedestals.
- **Memoized Geometry & Bounding Boxes:** Bounding box calculations and collision meshes are cached to eliminate garbage collection frame spikes.

3.4 Production Deployment Architecture (Vercel + Railway)

Full production readiness guidelines documented step-by-step in ``Docs/Deploy.txt``:

Tier	Platform	Build / Runtime Setup	Environment Configuration
Frontend	Vercel	Next.js 16 App Router build (<code>`npm run build`</code>). Automatic SSL, global CDN edge network.	<code>`NEXT_PUBLIC_BACKEND_URL`</code> pointing to deployed Railway backend URL.
Backend	Railway	Python 3.11 environment. Procfile runner: <code>`web: uvicorn main:app --host 0.0.0.0 --port \$PORT`</code> .	<code>`GROQ_API_KEY`</code> (Groq LLM key), <code>`FRONTEND_URL`</code> (Vercel origin for CORS).

4. RAG Curator API Contract (``POST /ask``)

HTTP Request (POST /ask)	HTTP Response (JSON)
// Request Payload (Next.js Frontend -> FastAPI Backend) { "session_id": "a1b2c3d4-e5f6-7890", "question": "What is the historical significance of this exhibit?", "artifact_id": "ART001", "tone": "educational" }	// Response Payload (FastAPI Backend -> Next.js Frontend) { "answer": "The Old Arrow Maker was created by Edmonia Lewis in 1872...", "source": { "artifact": "Old Arrow Maker", "gallery": "History Gallery" }, "confidence": 0.842, "evidence": "Old Arrow Maker. Sculptor: Edmonia Lewis . Date: 1872...", "refused": false, "session_id": "a1b2c3d4-e5f6-7890", "tone": "educational", "related_suggestions": [{ "artifact_id": "ART002", "artifact_name": "Benin Bronze", "similarity": 0.63 }] }

5. Updated Project File Structure

virtual-museum/ ■■■■ src/ ■ ■■■■ app/ # Next.js App Router pages & layout ■ ■■■■ components/ ■ ■ ■■■■ curator/ ■ ■ ■ ■ ■ AICuratorPanel.jsx # AI Curator UI + TTS Voice Button (Phase 4 & 5) ■ ■ ■■■■ ar/ ■ ■ ■ ■ ■ AROverlayUI.jsx # WebXR AR DOM overlay + Curator Integration (Phase 5) ■ ■ ■■■■ museum/ ■ ■ ■■■■ controls/ # 3D Museum Gallery Canvas & White Ceiling (Phase 0 & 5) ■ ■ ■■■■ artifacts/ # WASD + 360° Mouse Pointer Lock Controls (Phase 5) ■ ■■■■ utils/ # 3D GLB Artifact Inspection components ■ ■ ■■■■ voice/ ■ ■ ■ ■ ■ textToSpeech.js # Web Speech API Synthesis Utility (Phase 5 - NEW) ■ ■ ■■■■ api/ ■ ■ ■■■■ curator.js # askCurator() API client ■ ■■■■ data/ # 15 Smithsonian exhibit metadata records ■■■■ backend/ ■ ■■■■ main.py # FastAPI app entry point & CORS configuration ■ ■■■■ Procfile # Railway backend start process command ■ ■■■■ requirements.txt # Python backend dependencies ■ ■■■■ rag/ # SentenceTransformers, NumPy RAG & Groq pipeline ■■■■ Docs/ ■ ■■■■ Deploy.txt # Complete Vercel + Railway Deployment Guide (NEW) ■ ■■■■ Voice.txt # Speech Synthesis Architecture Specifications (NEW) ■ ■■■■ Optimise AR.txt # Mobile AR Performance Benchmarks & Rules (NEW) ■■■■ README.md # Master Project Documentation & Feature Guide	
---	--

6. System Verification & Sign-Off

The Virtual Museum AR/VR system has undergone comprehensive end-to-end verification across desktop browsers and Android WebXR devices:

- ✓ **RAG Accuracy & Hallucination Prevention:** Tested against out-of-distribution questions; 0.50 threshold gate successfully triggers deterministic refusal without calling LLM tokens.

- ✓ **Voice Speech Synthesis (TTS):** Verified across Chrome, Edge, and Android WebView; audio speech stops instantly on panel close or AR exit.
- ✓ **3D & AR Rendering Performance:** Maintained locked 60 FPS in desktop mode and smooth 60 FPS plane tracking during mobile WebXR AR sessions.
- ✓ **Deployment Verification:** Backend FastAPI health check and Next.js frontend builds verified for production readiness.